

Go Javeiros (Grupo 1)

Joniel Rodrigues de Oliveira - TL

Jasmine de Sá Araújo

André Luiz Alves da Silva

Bruna Ribeiro

João José Rocha de Sousa De Sousa

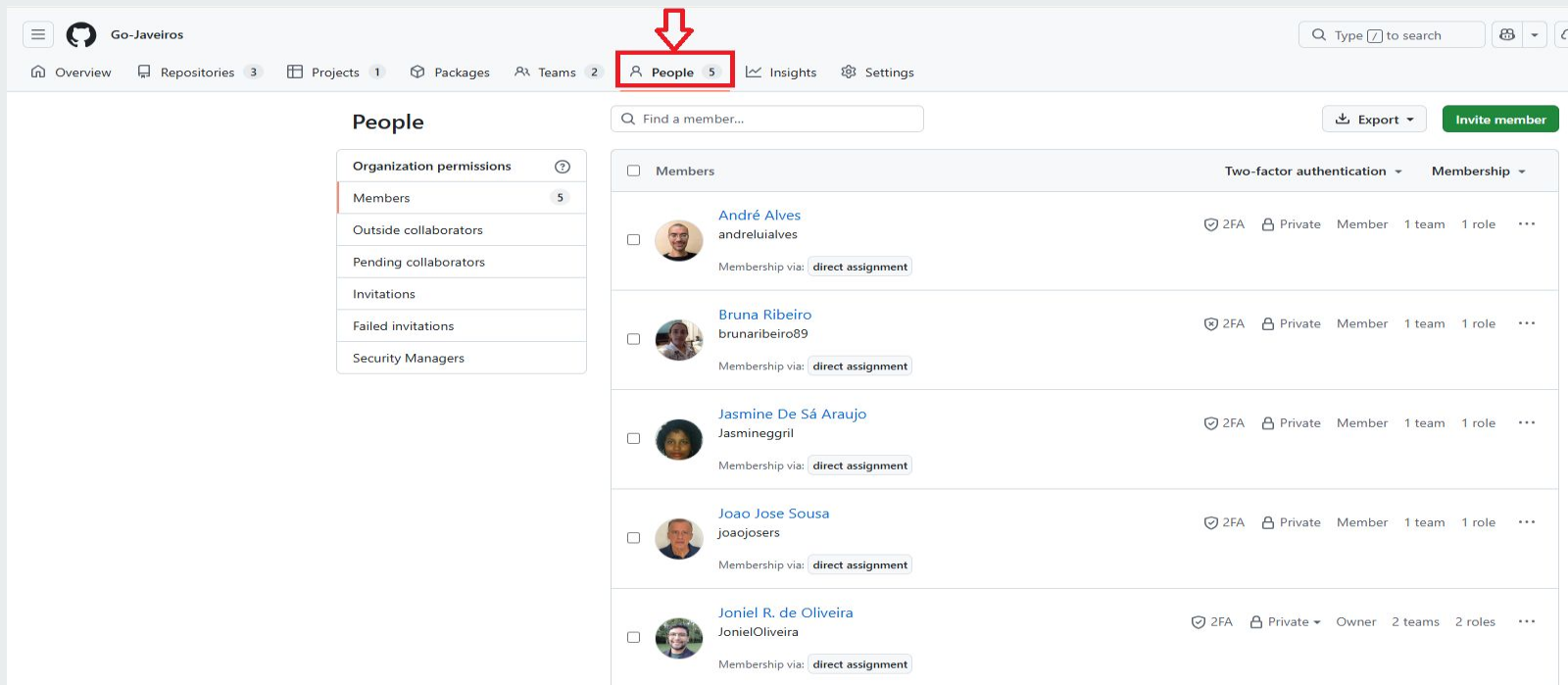


Parte 1 – Organização do grupo:

- [01] – Definição dos membros da organização
- [02] – Definição dos times da organização
- [03] – Definição das issues do repositório
- [04] – Definição dos responsáveis pelas issues
- [05] – Definição do projeto no GitHub
- [06] – Definição da ferramenta de gestão
- [07] – Definição do fluxo de trabalho no repositório
- [08] – Definição padrão de commits recomendado
- [09] – Revisão - definição de testes unitários
- [10] – Revisão - aceitação de PR (Pull Request)
- [11] – Revisão - código do Workflow do GitHub Actions



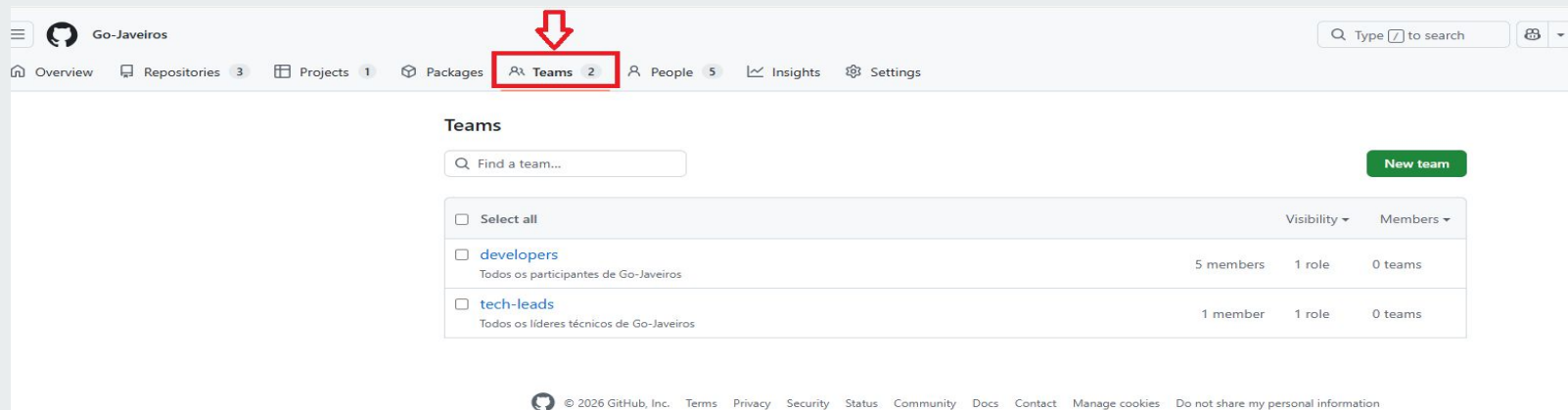
[01] Definição dos membros da organização:



The screenshot displays the GitHub interface for the organization 'Go-Javeiros'. The top navigation bar includes links for Overview, Repositories (3), Projects (1), Packages, Teams (2), **People (5)**, Insights, and Settings. A red arrow points to the 'People' tab. On the left, the 'People' sidebar shows 'Organization permissions' with 'Members' (5) selected, and other options like 'Outside collaborators', 'Pending collaborators', 'Invitations', 'Failed invitations', and 'Security Managers'. The main content area shows a list of members with search and export options.

Members	Two-factor authentication	Membership
☐ André Alves andreluialves Membership via: direct assignment	2FA Private Member 1 team 1 role ...	
☐ Bruna Ribeiro brunaribeiro89 Membership via: direct assignment	2FA Private Member 1 team 1 role ...	
☐ Jasmine De Sá Araujo Jasmineggri1 Membership via: direct assignment	2FA Private Member 1 team 1 role ...	
☐ Joao Jose Sousa joajosers Membership via: direct assignment	2FA Private Member 1 team 1 role ...	
☐ Joniel R. de Oliveira JonielOliveira Membership via: direct assignment	2FA Private Owner 2 teams 2 roles ...	

[02] Definição dos times da organização:



The screenshot shows the GitHub interface for the organization 'Go-Javeiros'. The navigation bar at the top includes links for Overview, Repositories (3), Projects (1), Packages, Teams (2), People (5), Insights, and Settings. The 'Teams' link is highlighted with a red box and a red arrow. Below the navigation bar, the 'Teams' section is displayed, featuring a search bar and a 'New team' button. A table lists the existing teams:

☐ Select all	Visibility ▾	Members ▾
☐ developers Todos os participantes de Go-Javeiros	5 members	1 role
☐ tech-leads Todos os líderes técnicos de Go-Javeiros	1 member	1 role

At the bottom of the page, the footer includes the GitHub logo, copyright information (© 2026 GitHub, Inc.), and links for Terms, Privacy, Security, Status, Community, Docs, Contact, Manage cookies, and Do not share my personal information.

[02] Definição dos times da organização:



[03] Definição das issues do repositório:

 Go-Javeiros / tecnicas-programacao ▾

Q Type to search  

<> Code Issues 2 Pull requests 2 Agents Actions Projects Wiki Security and quality Insights Settings

is:issue state:closed   Labels Milestones New issue

☐ Open 2 ☒ Closed 11

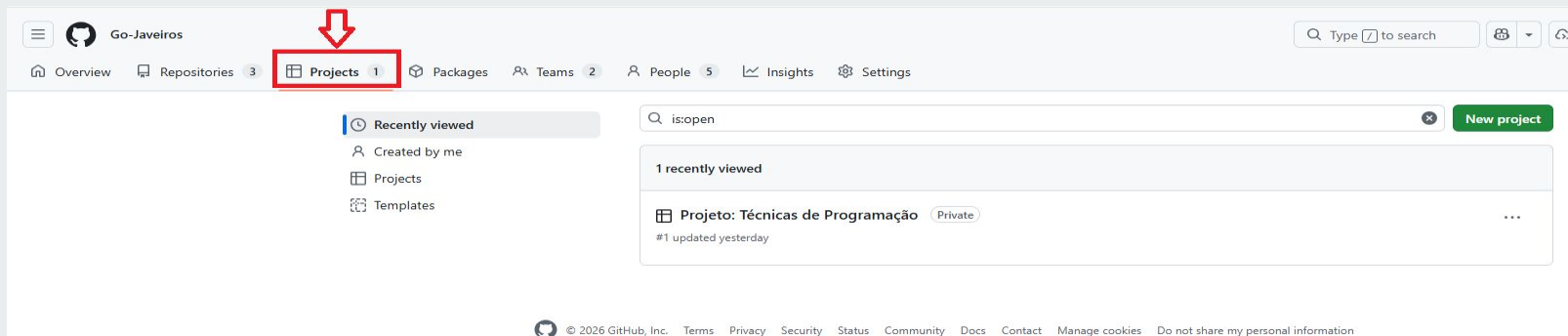
Author ▾ Labels ▾ Projects ▾ Milestones ▾ Assignees ▾ Types ▾  Newest ▾

☐	☒	Exercício 7 — Programação Paralela (Módulo 7) enhancement	Feature #14 · by JonielOliveira was closed yesterday ·  v1.0.0	
☐	☒	Exercício 3b — Streams (Módulo 3) enhancement	Feature #11 · by JonielOliveira was closed yesterday ·  v1.0.0	 1
☐	☒	Exercício 3a — Streams (Módulo 3) enhancement	Feature #10 · by JonielOliveira was closed yesterday ·  v1.0.0	 1
☐	☒	Exercício 6 — Optional (Módulo 6) enhancement	Feature #9 · by JonielOliveira was closed yesterday ·  v1.0.0	
☐	☒	Exercício 1c — Datas (Módulo 1) enhancement	Feature #8 · by JonielOliveira was closed yesterday ·  v1.0.0	 1
☐	☒	Exercício 1b — Datas (Módulo 1) enhancement	Feature #7 · by JonielOliveira was closed yesterday ·  v1.0.0	 1
☐	☒	Exercício 1a — Datas (Módulo 1) enhancement	Feature #6 · by JonielOliveira was closed yesterday ·  v1.0.0	 1
☐	☒	Exercício 2 — Programação Funcional (Módulo 2) enhancement	Feature #5 · by JonielOliveira was closed 4 days ago ·  v1.0.0	 1
☐	☒	Exercício 4 — NIO.2 (Módulo 4) enhancement	Feature #4 · by JonielOliveira was closed 4 days ago ·  v1.0.0	 1
☐	☒	Exercício 5b — IO Streams (Módulo 5) enhancement	Feature #3 · by JonielOliveira was closed 4 days ago ·  v1.0.0	 1
☐	☒	Exercício 5a — IO Streams (Módulo 5) enhancement	Feature #2 · by JonielOliveira was closed 4 days ago ·  v1.0.0	 1

[04] Definição dos responsáveis pelas issues:

Issue	Exercício	Tópico	Módulo	Arquivo	Responsável
#2	Exercício 5a	IO Streams	5	ArquivoMotor.java	André
#3	Exercício 5b	IO Streams	5	ArquivoMotor.java	André
#4	Exercício 4	NIO.2	4	ArquivoMotor.java	André
#5	Exercício 2	Programação Funcional	2	Repositorio.java	João
#6	Exercício 1a	Datas	1	Emprestimo.java	Bruna
#7	Exercício 1b	Datas	1	Emprestimo.java	Bruna
#8	Exercício 1c	Datas	1	Emprestimo.java	Bruna
#9	Exercício 6	Optional	6	BibliotecaServico.java	Joniel
#10	Exercício 3a	Streams	3	LivroRepositorio.java	João
#11	Exercício 3b	Streams	3	LivroRepositorio.java	João
#12	Exercício 3c	Streams	3	RelatorioServico.java	Jasmine
#13	Exercício 3d	Streams	3	RelatorioServico.java	Jasmine
#14	Exercício 7	Programação Paralela	7	RelatorioServico.java	Joniel

[05] Definição do projeto no GitHub:



The screenshot shows the GitHub profile page for 'Go-Javeiros'. The 'Projects' tab is highlighted with a red box and a red arrow pointing to it. The page displays a search bar with 'is:open' and a 'New project' button. Below the search bar, there is a section titled '1 recently viewed' showing a project named 'Projeto: Técnicas de Programação' with a 'Private' label and a status of '#1 updated yesterday'.

Go-Javeiros

Overview Repositories 3 **Projects 1** Packages Teams 2 People 5 Insights Settings

is:open New project

1 recently viewed

Projeto: Técnicas de Programação Private ...

#1 updated yesterday

© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

[06] Definição da ferramenta de gestão:

Projeto: Técnicas de Programação

Views: Backlog | Priority board | Team items | Roadmap | My items | + New view

Filter by keyword or by field

Backlog 0 / 5 | Estimate: 0 | This item hasn't been started

Ready 0 | Estimate: 0 | This is ready to be picked up

In progress 2 / 3 | Estimate: 10 | This is actively being worked on

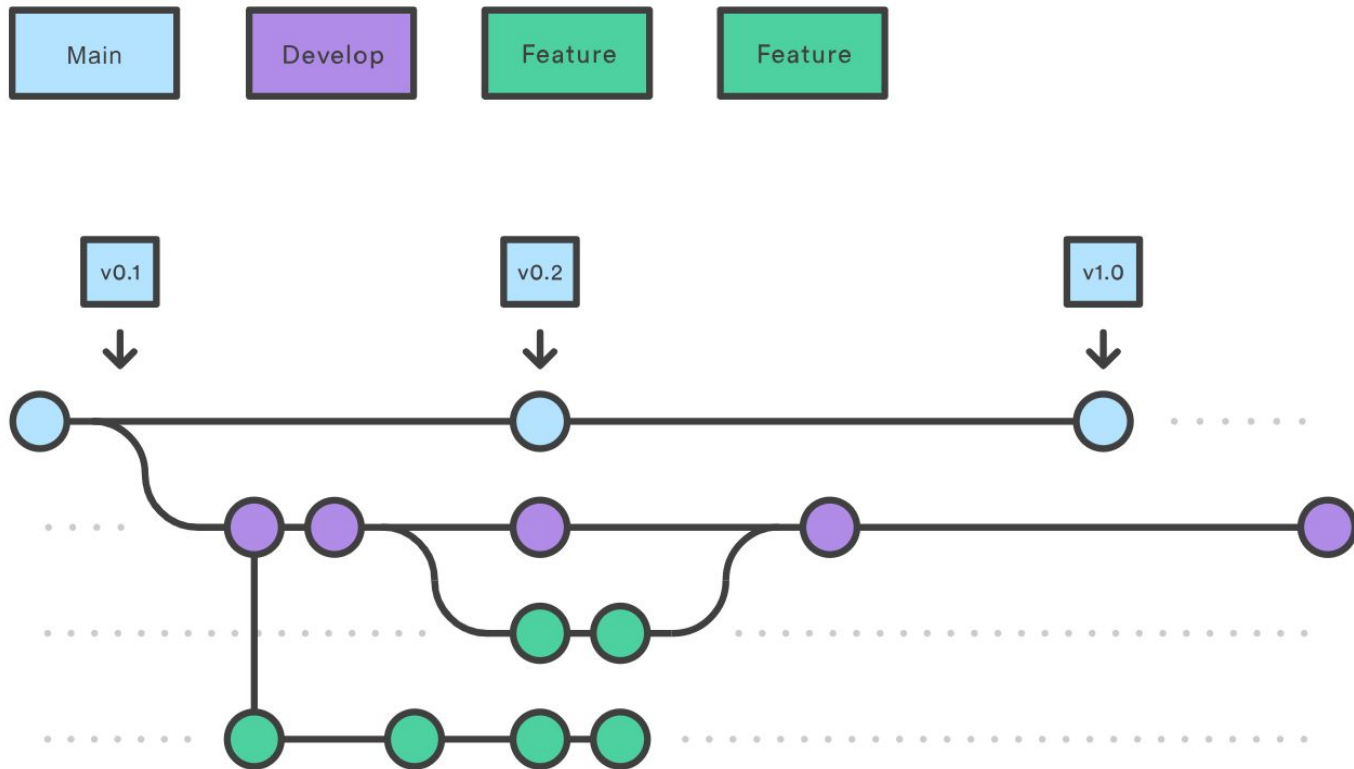
- tecnicas-programacao #12
Exercício 3c — Streams (Módulo 3)
PO S M
- tecnicas-programacao #13
Exercício 3d — Streams (Módulo 3)
PO S M

In review 0 / 5 | Estimate: 0 | This item is in review

Done 17 | Estimate: 53 | This has been completed

- tecnicas-programacao #9
Exercício 6 — Optional (Módulo 6)
PO M
- tecnicas-programacao #6
Exercício 1a — Datas (Módulo 1)
#16 PO S M
- tecnicas-programacao #10
Exercício 3a — Streams (Módulo 3)
#19 PO S M
- tecnicas-programacao #11
Exercício 3b — Streams (Módulo 3)
#19 PO S M
- tecnicas-programacao #5
Exercício 2 — Programação Funcional (Módulo 2)
#17 PO S M
- tecnicas-programacao #8
Exercício 1c — Datas (Módulo 1)
#16 PO S M

[07] Definição do fluxo de trabalho no repositório:



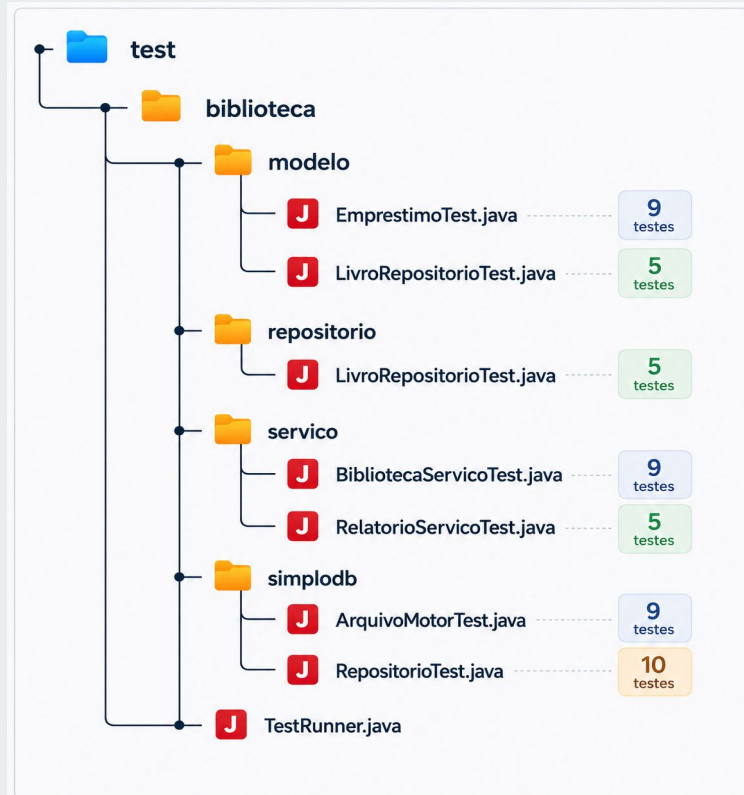
[08] Definição padrão de commits recomendado:

📄 Padrão de Commits

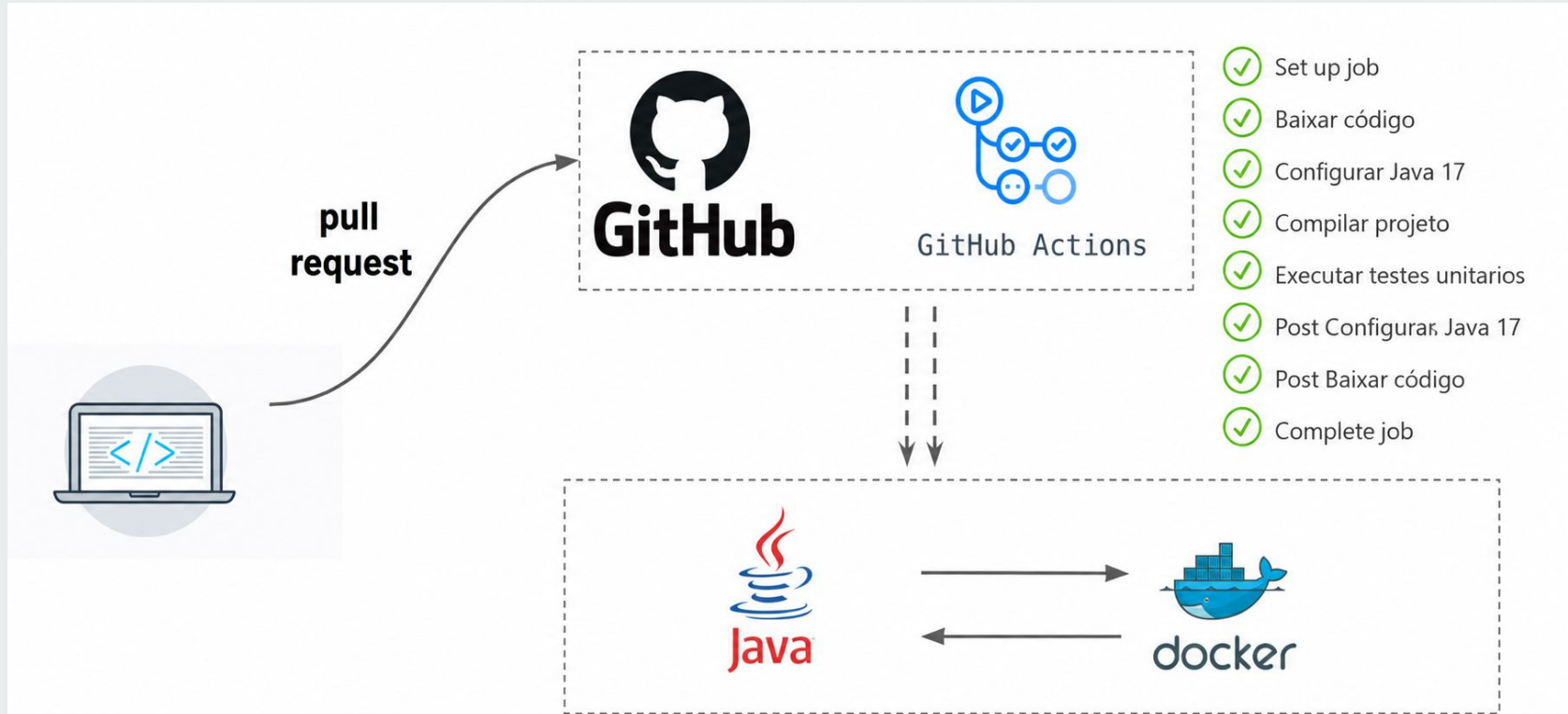
Nosso repositório segue o padrão [Conventional Commits](#) adaptado com **emojis** para facilitar a identificação do tipo de mudança.

Emoji	Código	Tipo	Descrição	Exemplo de uso
✨	:sparkles:	feat	Nova funcionalidade adicionada	✨ feat(auth): adicionar login com Google
🐛	:bug:	fix	Correção de bug	🐛 fix(api): corrigir erro 500 ao criar usuário
📖	:books:	docs	Alterações na documentação	📖 docs(readme): adicionar instruções de instalação
🎨	:art:	style	Mudanças de formatação/estilo (sem impacto no código)	🎨 style: padronizar indentação com Prettier
♻️	:recycle:	refactor	Refatoração de código sem alteração de comportamento	♻️ refactor(user-service): extrair validações
⚡	:zap:	perf	Melhorias de performance	⚡ perf(query): otimizar busca por índice
✅	:white_check_mark:	test	Adição ou modificação de testes	✅ test(auth): adicionar teste de refresh token
🔧	:hammer_and_wrench:	build	Mudanças em build, dependências ou ferramentas	🔧 build: atualizar express para ^4.19.0
🤖	:robot:	ci	Alterações em configuração de CI/CD	🤖 ci: adicionar workflow de testes no GitHub
🚚	:truck:	chore	Tarefas diversas sem impacto no código funcional (scripts, limpeza, etc.)	🚚 chore: remover arquivos temporários
⏮️	:rewind:	revert	Reversão de commit anterior	⏮️ revert: "feat(auth): adicionar refresh token"
🚀	:rocket:	release	Marcação de versão/release	🚀 release: v1.0.0

[09] Revisão - definição de testes unitários:



[10] Revisão - aceitação de PR (Pull Request):



14

The image shows a Windows desktop with a VS Code editor open. The editor has two tabs: 'ci.yml' and 'jobs'. The 'ci.yml' file contains a GitHub Actions workflow for building, testing, and deploying a Java application. The 'jobs' file contains a GitHub Actions workflow for validating the application. The workflow includes steps for building the project, running unit tests, and comparing the output with expected results.

```
github> workflows > | ci.yml
1  name: CI
2
3  on:
4    pull_request:
5      branches:
6        - develop
7        - main
8    push:
9      branches:
10       - develop
11       - main
12
13  jobs:
14    build-develop:
15      name: Build - develop
16      if: github.base_ref == 'develop' || github.ref == 'refs/heads/develop'
17      runs-on: ubuntu-latest
18
19      steps:
20        - name: Baixar código
21          uses: actions/checkout@v4
22
23        - name: Configurar Java 17
24          uses: actions/setup-java@v4
25          with:
26            distribution: temurin
27            java-version: '17'
28
29        - name: Compilar projeto
30          run: |
31            mkdir -p out
32            javac -encoding UTF-8 -d out \
33              src/simulob/*.java \
34              src/biblioteca/modelo/*.java \
35              src/biblioteca/repositorio/*.java \
36              src/biblioteca/servico/*.java \
37              src/Main.java
38
39        - name: Executar testes unitarios
40          run: |
41            javac -encoding UTF-8 -cp out -d out $(find test -name '*.java')
42            java -Dfile.encoding=UTF-8 -cp out TestRunner
43
44    validate-main:
45      name: Validate - main
46      if: github.base_ref == 'main' || github.ref == 'refs/heads/main'
47      runs-on: ubuntu-latest
48
49      steps:
50        - name: Baixar código
51          uses: actions/checkout@v4
52
53        - name: Configurar Java 17
54          uses: actions/setup-java@v4
55          with:
56            distribution: temurin
57            java-version: '17'
58
59github> workflows > | jobs
60
61 jobs:
62   validate-main:
63     steps:
64       - name: Configurar Java 17
65         with:
66           - name: Compilar projeto
67             run: |
68               mkdir -p out
69               javac -encoding UTF-8 -d out \
70                 src/simulob/*.java \
71                 src/biblioteca/modelo/*.java \
72                 src/biblioteca/repositorio/*.java \
73                 src/biblioteca/servico/*.java \
74                 src/Main.java
75
76       - name: Executar testes unitarios
77         run: |
78           javac -encoding UTF-8 -cp out -d out $(find test -name '*.java')
79           java -Dfile.encoding=UTF-8 -cp out TestRunner
80
81       - name: Executar Main
82         run: |
83           rm -rf db
84           mkdir -p db
85           java -Dfile.encoding=UTF-8 -cp Main > output.txt
86
87       - name: Comparar saída esperada ignorando datas e listas
88         run: |
89           sed -E \
90             -e 's/[0-9](2)\.[0-9](2)\.[0-9](4) [0-9](2):[0-9](2)<DATA_HORA>/g' \
91             -e 's/Romance: \[([^\]]*)\]/Romance: <LISTA_IGNOREADA>/g' \
92             -e 's/Poesia: \[([^\]]*)\]/Poesia: <LISTA_IGNOREADA>/g' \
93             -e 's/Top 5 mais emprestados: \[([^\]]*)\]/Top 5 mais emprestados: <LISTA_IGNOREADA>/g' \
94             -e 's/Gêneros disponíveis: \[([^\]]*)\]/Gêneros disponíveis: <LISTA_IGNOREADA>/g' \
95             expected/expected-output.txt > expected-normalized.txt
96
97           sed -E \
98             -e 's/[0-9](2)\.[0-9](2)\.[0-9](4) [0-9](2):[0-9](2)<DATA_HORA>/g' \
99             -e 's/Romance: \[([^\]]*)\]/Romance: <LISTA_IGNOREADA>/g' \
100            -e 's/Poesia: \[([^\]]*)\]/Poesia: <LISTA_IGNOREADA>/g' \
101            -e 's/Top 5 mais emprestados: \[([^\]]*)\]/Top 5 mais emprestados: <LISTA_IGNOREADA>/g' \
102            -e 's/Gêneros disponíveis: \[([^\]]*)\]/Gêneros disponíveis: <LISTA_IGNOREADA>/g' \
103            output.txt > output-normalized.txt
104
105           if [ diff -u --strip-trailing-cr expected-normalized.txt output-normalized.txt; then
106             echo "Saída do programa não corresponde à saída esperada."
107             exit 1
108           fi
```

Parte 2 – Implementação do projeto:

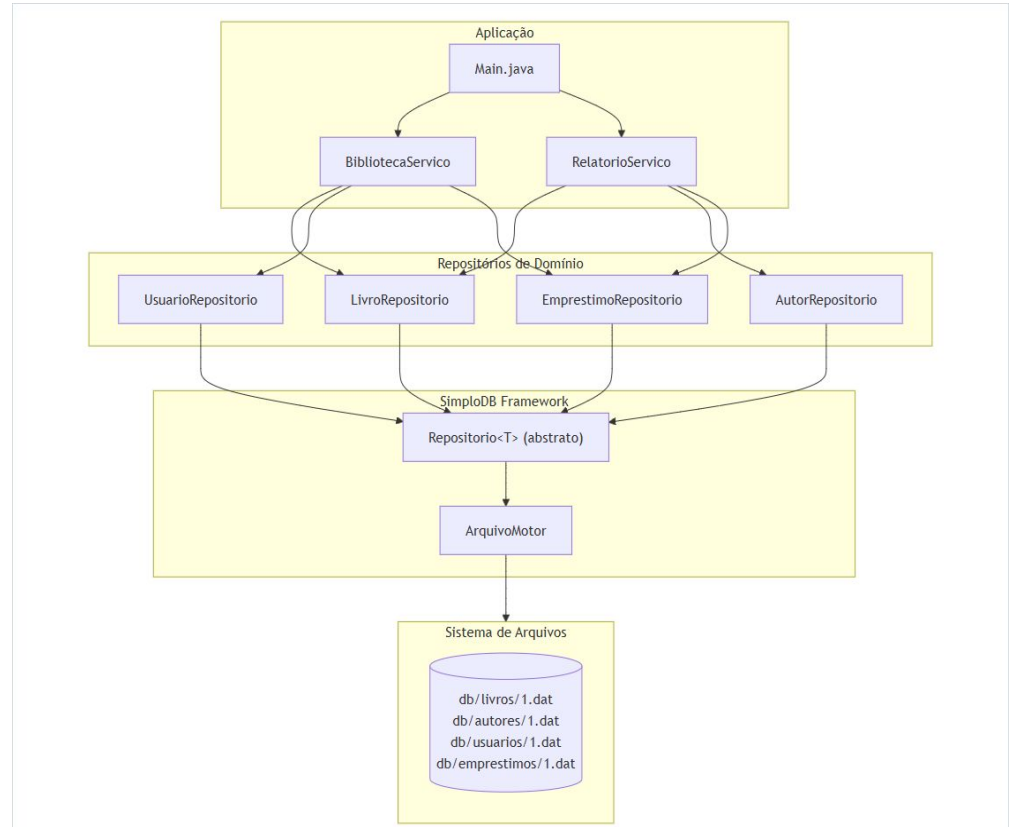
- [12] – Biblioteca Digital
- [13] – Arquitetura e decisões técnicas
- [14] – Conceitos aplicados - Programação Funcional
- [15] – Conceitos aplicados - Streams API - filter() - Ordenação
- [16] – Conceitos aplicados - Streams API - Agrupamento groupingBy()
- [17] – Conceitos aplicados - IO Streams
- [18] – Dificuldades e desafios
- [19] – Aprendizados
- [20] – Demonstração
- [21] – QR CODE do repositório



[12] Biblioteca Digital:

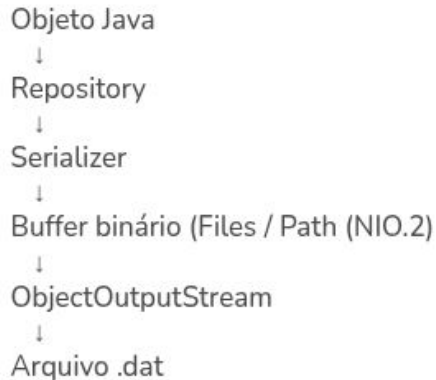
O projeto desenvolvido é uma biblioteca digital utilizando o SimploDB, uma camada de persistência própria inspirada no *Spring Data JPA**, capaz de armazenar objetos Java diretamente em arquivos.

*O *Spring Data JPA* é um framework do ecossistema Spring que simplifica a comunicação de aplicações Java com bancos de dados relacionais



[13] Arquitetura e decisões técnicas:

Como o SimploDB funciona: o caminho de um objeto desde o salvar() até o arquivo .dat



[13] Arquitetura e decisões técnicas:



Por que serialização?

Em Java, os objetos são armazenados apenas na memória RAM, que é volátil. Isso significa que, ao encerrar a execução do programa, todos os dados são perdidos. A serialização permite gravar o estado dos objetos em arquivos, garantindo sua persistência além do ciclo de vida da aplicação.

Quais as vantagens e limitações em relação a um banco de dados real?

Vantagens:

Simplicidade: implementação rápida, com poucas linhas de código. **Persistência imediata:** ideal para projetos pequenos ou protótipos. **Independência de infraestrutura:** não requer instalação ou configuração de um banco de dados

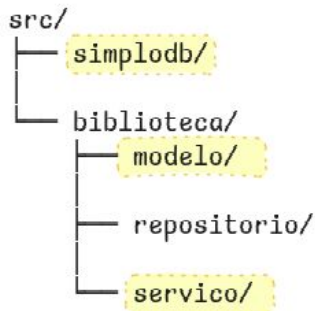
Desvantagens:

Baixa eficiência em grandes volumes: a busca se torna lenta quando há muitos registros. **Atualizações complexas:** modificar um objeto exige reescrever o arquivo inteiro. **Fragilidade diante de mudanças:** alterações na estrutura das classes podem tornar objetos previamente serializados incompatíveis e ilegíveis.

[13] Arquitetura e decisões técnicas:

Qual foi a parte mais interessante da arquitetura para o grupo?

A separação em camadas, dividindo entidades (*model*) e regras de negócio (*service*), utilizando o SimpleDB para persistência de dados (*db*).



[14] Conceitos aplicados - Programação Funcional

- Programação Funcional Interface Predicate<T>
- aceita Predicate <T> - função que retorna tipo Boolean para cada elemento.
- Usa expressões lambdas que geram código sucinto

Implementação no Projeto em Repositorio Java e LivroRepositorio:

```
public Optional<Livro> buscarPorIsbn(String isbn) {  
  
    return buscarComFiltro(i -> i.getIsbn().equal  
  
        .stream()  
  
        .findFirst()  
  
}
```

[15] Conceitos aplicados - Streams API - filter() - Ordenação

Streams API Busca filter() e Ordenação

- Implementação no Projeto

```
public List<Livro> buscarPorGeneroCoordenado (String genero) {  
  
    return buscarTodos().stream()  
        .filter( l -> l.getGenero().equalsIgnoreCase(genero))  
        .sorted(Comparator.comparingInt  
            (livro::getAnoPublicacao)  
            .reversed())  
        .collect(Collectors.toList());  
  
}
```

[16] Conceitos aplicados - Streams API - Agrupamento groupingBy()

Streams API Agrupamento por gênero

- Implementação no Projeto

```
publicMap<String, List<Livro>> agruparPorGenero()
{
    return buscarTodos().stream()
        .collect(Collectors.groupingBy(
            livro::getGenero));
}
```

[17] Conceitos aplicados - IO Streams:

Uso no método salvar() da classe ArquivoMotor:

```
try (OutputStream oos =  
    new ObjectOutputStream(Files.newOutputStream(caminho))) {  
    oos.writeObject(obj);  
}
```

`Files.newOutputStream(caminho)` cria um fluxo de saída (`OutputStream`) para o arquivo.

`ObjectOutputStream` envolve esse fluxo e permite escrever objetos Java no arquivo.

`oos.writeObject(obj)` serializa o objeto `obj` e salva seus dados no disco.

O bloco `try-with-resources` fecha o stream automaticamente após o uso, evitando vazamento de recursos.

[18] Dificuldades e desafios:



Qual exercício deu mais trabalho? Por quê?

O grupo chegou à conclusão de que todos os exercícios tiveram seus desafio e dificuldades de forma equilibrada.

Algum erro difícil de depurar? Como resolveram?

O grupo não encontrou nenhum erro que fosse difícil de depurar, durante o desenvolvimento dos exercícios.

O que fariam diferente se recomeçasse?

O que poderíamos fazer de forma diferente seria a resolução dos exercícios utilizando Pair programming .

[19] Aprendizados:



O que o grupo considera o aprendizado mais importante do projeto:

Trabalho em equipe, simulando um ambiente mais próximo da realidade. Alinhar ideias em tempo hábil para a entrega do projeto.

Como os conceitos do curso se conectam no mundo real:

São conceitos comuns usados no dia a dia de desenvolvimento de software.

Ex.: *Streams API*: facilita o processamento eficiente de listas. / *Datas*: essenciais para agendamentos de tarefas e validade de tokens. / *IO Streams*: permitem leitura/escrita de arquivos, fundamentais para exportação de relatórios.

Projeto: simulação de arquitetura de software real de forma simplificada.

[20] Demonstração:

Valeu! Vamos ver o projeto sendo executado.



[21] QR CODE do repositório:



